Appending and Reversing Linked Lists

Kevin Browne

How could we append a list to another list?

Appending lists

- If we alter the tail of one list to point to the head of another, we will have appended a list
- We can traverse the first list until we reach the tail
- And then we can set the tail to point to the head of the second list
- If the first list is empty, then "appending" the 2nd list is a matter of returning the head of the 2nd list

Append list function

```
Node *append_list(Node *head1, Node *head2)
{
  if (head1 == NULL) return head2;

  Node *current = head1;
  while (current->next != NULL) current = current->next;
  current->next = head2;
  return head1;
}
```

Testing the append list function

```
// Test the append_list function
Node *list1 = NULL, *list2 = NULL;
for (int i = 1; i <= 3; i++) list1 = insert_at_tail(list1, i);
for (int i = 4; i <= 10; i++) list2 = insert_at_tail(list2, i);
printf("\nList 1:\n");
print_list(list1);
printf("\nList 2:\n");
print_list(list2);
list1 = append_list(list1, list2);
printf("\nList 1 after appending List 2:\n");
print_list(list1);
```

Append list function test results…

```
List 1:
Node 0: 1
Node 1: 2
Node 2: 3

List 2:
Node 0: 4
Node 1: 5
Node 2: 6
Node 3: 7
Node 4: 8
Node 5: 9
Node 6: 10
```

Append list function test results…

```
List 1 after appending List 2:
Node 0: 1
Node 1: 2
Node 2: 3
Node 3: 4
Node 4: 5
Node 5: 6
Node 6: 7
Node 7: 8
Node 8: 9
Node 9: 10
```

Can we reverse a linked list?

Reversing a linked list

- If a list is empty or one node, it's already "reversed"
- Otherwise, can traverse the list, and reverse the pointers!
  - The head node would point to NULL, becoming the new tail
  - All other nodes can be set to point to the node previous to them in the linked list
- We could return the tail as the new head of the list

Reverse list function

```
Node *reverse_list(Node *head)
{
  if (head == NULL) return NULL;
  if (head->next == NULL) return head;

  Node *current = head;
  Node *next_node = head->next;

  current->next = NULL;

  while (next_node != NULL)
  {
    Node *tmp = next_node->next;
    next_node->next = current;
    current = next_node;
    next_node = tmp;
  }

  return current;
}
```

Test the reverse list function

```
// Test the reverse_list function
printf("\nList 1 before reversal:\n");
print_list(list1);
list1 = reverse_list(list1);
printf("\nList 1 after reversal:\n");
print_list(list1);
```

Reverse list function test results…

```
List 1 before reversal:
Node 0: 1
Node 1: 2
Node 2: 3
Node 3: 4
Node 4: 5
Node 5: 6
Node 6: 7
Node 7: 8
Node 8: 9
Node 9: 10
```

Reverse list function test results…

```
List 1 after reversal:
Node 0: 10
Node 1: 9
Node 2: 8
Node 3: 7
Node 4: 6
Node 5: 5
Node 6: 4
Node 7: 3
Node 8: 2
Node 9: 1
```